Name of the Student	Sham S.
Reg. No	192325076
GitHub Link	https://github.com/Sham-2005/MLA0403-Deep_learning

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning **COURSE CODE: MLA04**
SLOT :

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

Course Outcome Weightage: 10% **Duration: 90 minutes**
Assessment Tool Weightage: 40% **Mark : 25**

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:**

Criteria	Understanding of Industry Problem **(2)**	Application of Engineering Concepts **(2.5)**	Solution Design **(2.5)**	Use of Tools **(2)**	Analysis ***(1)***	**Total (10)**
Weightage	***20 %***	***25%***	***25%***	***20 %***	***10 %***	**100%**
Q1						
Q2						
Q3						
Q4						
Q5						

Total						

QUESTIONS: 05 **Total Marks:25**

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

CO4 – ASSESSMENT TOOL 2
Industry Problem Solving Task
Course: Deep Learning – MLA04

Q1. Transfer Learning for Skin Cancer Detection

1. Understanding of the Industry Problem

A healthcare startup wants to develop an AI system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, training a deep CNN from scratch may result in overfitting and poor generalization.

A **transfer learning approach** using a pre-trained CNN is therefore appropriate. The pre-trained model can reuse visual features learned from a large image dataset and adapt them to skin lesion classification.

The objective is to classify dermoscopic images into appropriate skin-lesion categories, such as **benign and malignant**, depending on the available labeled dataset.

2. Proposed Model

A pre-trained **EfficientNetB0** or **ResNet50** can be used. EfficientNet is a suitable choice because it provides a good balance between classification performance and computational efficiency.

Architecture

Dermoscopic Image
↓
Image Preprocessing
↓
Resize to 224 × 224
↓
Data Augmentation
↓
Pre-trained EfficientNetB0
↓
Global Average Pooling
↓
Dense Layer
↓
Dropout
↓
Softmax / Sigmoid Output
↓
Skin Lesion Classification

3. Transfer Learning Strategy

Stage 1 – Freeze the Backbone

Initially, all convolutional layers of the pre-trained network are frozen.
The newly added classification layers are trained using the available dermoscopic images.
This allows the model to use previously learned general visual features such as:

- Edges
- Shapes
- Textures
- Patterns

Stage 2 – Fine-Tuning

After the new classification head becomes stable, the last few convolutional blocks are unfrozen.
The model is then trained using a **very small learning rate**.
The early layers remain frozen because they contain general features, while deeper layers are fine-tuned to learn skin-lesion-specific features.

4. Training Strategy

The dataset can be divided into:

- 70% training
- 15% validation
- 15% testing

Data augmentation can include:

- Rotation
- Horizontal and vertical flipping
- Zoom
- Translation

- Brightness adjustment

Suitable training configuration:

- Optimizer: Adam
- Initial learning rate: 0.001
- Fine-tuning learning rate: 0.0001
- Batch size: 16 or 32
- Early stopping
- Learning-rate reduction

Class imbalance should also be considered because medical datasets may contain fewer malignant samples than benign samples. Class weights or balanced sampling can be used.

5. Tools

The system can be implemented using:

- Python
- TensorFlow/Keras or PyTorch
- OpenCV/PIL
- NumPy
- Scikit-learn
- Matplotlib

6. Performance Analysis

The model should be evaluated using:

- Accuracy
- Precision
- Recall/Sensitivity
- Specificity
- F1-score
- ROC-AUC
- Confusion Matrix

In healthcare, **recall/sensitivity for malignant cases is particularly important**, because failing to identify a potentially cancerous lesion can be more serious than generating an additional false positive.

7. Justification

Transfer learning reduces the requirement for a very large labeled dataset. The pre-trained CNN provides useful low-level and high-level visual features, while fine-tuning allows the network to adapt to dermoscopic patterns.

Therefore, the proposed approach can provide better generalization, faster training, and reduced overfitting compared with training an entire CNN from scratch.

Q2. DenseNet and PixelNet for Smart-City Road Scene Segmentation

1. Understanding of the Industry Problem

A smart-city project requires an automated road-scene analysis system that identifies **roads, pedestrians, vehicles, and traffic signs at pixel level**. This is a semantic segmentation problem because every pixel must be assigned to an appropriate class.

The major requirements are:

- Accurate pixel-level segmentation
- Efficient feature extraction
- Good handling of complex road scenes
- Reasonable computational efficiency

The question specifically requires a combination of **DenseNet and PixelNet**.

2. Proposed Architecture

DenseNet can be used as the feature-extraction backbone, while PixelNet-style pixel-level processing can be used for classification of individual pixels.

Road Scene Image

↓

Preprocessing

↓

DenseNet Backbone

↓

Dense Feature Extraction

↓

Feature Fusion

↓

Pixel-Level Feature Representation

↓

PixelNet Segmentation Head

↓

1 × 1 Convolution

↓

Softmax

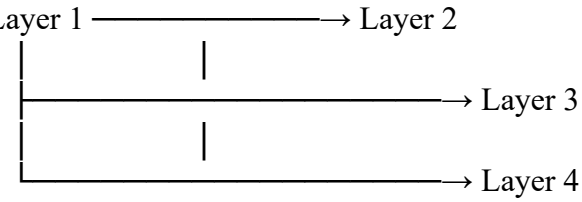
↓

Pixel-wise Segmentation Map

3. DenseNet Feature Extraction

DenseNet contains dense connections between layers.

Instead of each layer receiving information only from the previous layer, a layer can receive feature maps from earlier layers.



This allows features such as edges, textures, shapes, and object patterns to be reused.

For road-scene analysis, this can help preserve useful visual information across the network.

4. Pixel-Level Segmentation

The extracted DenseNet features are passed to the segmentation component.

Each pixel is classified into one of the required categories:

- Road
- Pedestrian
- Vehicle
- Traffic sign
- Background

The output is a segmentation mask in which each pixel contains a class label.

For example:

Input Image

↓

Feature Extraction

↓

Pixel Classification

↓
Segmentation Mask

Road → Class 1
Pedestrian → Class 2
Vehicle → Class 3
Traffic Sign→ Class 4
Background → Class 5

5. Improving Segmentation Accuracy

The proposed system can improve accuracy through:

Dense Feature Reuse

Dense connections allow earlier features to be reused by later layers.

Multi-Level Features

Low-level features help detect boundaries, while deeper features provide semantic information.

Pixel-Level Prediction

PixelNet-style processing allows the model to classify individual pixels rather than only predicting a single label for the complete image.

Data Augmentation

Training images can be augmented using:

- Rotation
- Cropping
- Scaling
- Horizontal flipping
- Brightness variation

6. Computational Efficiency

To improve efficiency:

- Use a lightweight DenseNet variant where appropriate.
- Use batch processing during training.
- Resize input images to a manageable resolution.
- Use GPU acceleration.
- Use 1 × 1 convolution for reducing feature dimensions.
- Apply model optimization during deployment.

7. Tools

Possible tools include:

- Python
- PyTorch or TensorFlow
- OpenCV
- NumPy
- CUDA-enabled GPU
- Matplotlib

Datasets containing annotated road scenes can be used for training and validation.

8. Performance Analysis

The system can be evaluated using:

- Pixel Accuracy
- Mean Intersection over Union (mIoU)
- Precision
- Recall

- F1-score
- Per-class IoU

The IoU of each important class should be separately analyzed because pedestrians and traffic signs may occupy much smaller regions than roads.

9. Justification

DenseNet provides effective feature reuse, while pixel-level classification provides detailed segmentation. Combining the two allows the system to identify objects and road regions accurately at pixel level while reducing unnecessary repeated feature learning.

Q3. Bidirectional RNN Encoder–Decoder for a Multilingual Chatbot

1. Understanding of the Industry Problem

A multinational customer-service company wants a multilingual chatbot capable of translating customer queries while preserving their contextual meaning.

The input and translated output can have different sequence lengths. Therefore, an **Encoder–Decoder sequence-to-sequence architecture** is appropriate.

The question specifically requires the Encoder–Decoder model to use **Bidirectional RNNs**.

2. Proposed Architecture

Customer Query
↓
Text Cleaning
↓
Tokenization
↓
Embedding
↓
Bidirectional RNN Encoder
↓
Context Representation
↓
RNN/LSTM Decoder
↓
Dense Layer
↓
Softmax
↓
Translated Query

3. Encoder

The encoder reads the source sentence and converts it into a sequence of hidden representations.

For example:

Input:

"Where is my order?"

The sentence is tokenized:

Where → is → my → order

The embedding layer converts the tokens into numerical vectors.

The Bidirectional RNN processes the sentence in two directions.

Forward Direction

Where → is → my → order

Backward Direction

order → my → is → Where

The outputs from both directions are combined.

4. Why Bidirectional Processing Helps

A standard RNN primarily processes information in one direction.

A Bidirectional RNN can use:

- Previous context
- Future context

This is useful because the meaning of a word may depend on words appearing later in the sentence.

For example:

"The customer did not receive the package."

The meaning of "receive" is influenced by the surrounding words.

Bidirectional processing provides the encoder with a more complete representation of the input sentence.

5. Decoder

The decoder generates the translated sentence one token at a time.

<START>
↓
Translated Word 1
↓
Translated Word 2
↓
Translated Word 3
↓
...
↓
<END>

The decoder uses the encoder's contextual representation to generate the target-language sequence.

6. Training Strategy

The training dataset contains multilingual sentence pairs:

Source Sentence → Target Sentence

Preprocessing includes:

1. Text cleaning
2. Tokenization
3. Vocabulary creation
4. Integer encoding
5. Padding
6. Adding START and END tokens

Teacher forcing can be used during training.

Suitable configuration:

- Adam optimizer
- Cross-Entropy loss
- Dropout
- Early stopping
- Learning-rate scheduling

7. Tools

- Python
- TensorFlow/Keras or PyTorch
- NumPy
- Pandas
- NLP tokenization libraries
- GPU acceleration

8. Performance Analysis

Translation quality can be measured using:

- Validation loss
- Token accuracy
- BLEU score
- Human evaluation

The system should also be tested using customer queries containing different sentence lengths and contextual expressions.

9. Justification

The Bidirectional RNN provides information from both directions of the input sequence. This creates a richer contextual representation for the Encoder–Decoder model and can improve translation accuracy, especially for sentences where word meaning depends strongly on surrounding context.

Q4. BPTT and LSTM for Stock Price Forecasting

1. Understanding of the Industry Problem

A financial analytics company wants to forecast stock prices using historical market data. Stock-market data is sequential because the value at one time can be related to information observed at previous time steps.

Typical input features can include:

- Opening price
- Closing price
- High price
- Low price
- Trading volume

The question requires an **LSTM-based prediction system** and an explanation of **Backpropagation Through Time (BPTT)**.

2. Input Representation

Suppose historical data from the previous 30 trading days is used.

Each day can be represented as:

[Open, High, Low, Close, Volume]

Therefore:

30 time steps × 5 features

can be supplied to the LSTM.

3. Proposed Architecture

Historical Market Data

↓

Data Cleaning

↓

Normalization

↓

Sliding Window Creation

↓
LSTM Layer
↓
Dropout
↓
LSTM / Dense Layer
↓
Dense Output
↓
Predicted Stock Price

A possible architecture is:

Input: 30 × 5
↓
LSTM – 64 Units
↓
Dropout – 0.2
↓
LSTM – 32 Units
↓
Dropout – 0.2
↓
Dense – 16 Units
↓
Dense – 1 Unit
↓
Predicted Closing Price

4. Data Preprocessing

The historical data should be:

1. Sorted chronologically.
2. Checked for missing values.
3. Normalized using Min-Max scaling or standardization.
4. Converted into fixed-length sequences.

For example:

Days 1–30 → Predict Day 31
Days 2–31 → Predict Day 32
Days 3–32 → Predict Day 33

This is called a **sliding-window approach**.

5. Backpropagation Through Time

BPTT is used to train recurrent neural networks by extending the network across its time steps.

For example:

Time 1 → Time 2 → Time 3 → Time 4 → Time 5
↓ ↓ ↓ ↓ ↓
h1 h2 h3 h4 h5

During training:

1. The input sequence is passed through the LSTM.
2. A prediction is produced.
3. The prediction is compared with the actual value.

4. The loss is calculated.
5. The error is propagated backward through the sequence.
6. LSTM weights are updated using the optimizer.

6. Why LSTM Instead of Traditional RNN?

Traditional RNNs can suffer from the **vanishing-gradient problem** when learning long-term dependencies.

Stock-market sequences may contain patterns across many time steps.

LSTM contains:

- Forget gate
- Input gate
- Output gate
- Cell state

These mechanisms help the model preserve useful information and discard irrelevant information.

Therefore, LSTM is generally more suitable than a traditional RNN for learning longer temporal dependencies.

7. Training Strategy

Suitable configuration:

- Optimizer: Adam
- Loss: Mean Squared Error
- Batch size: 32
- Early stopping
- Learning-rate scheduling
- Dropout

The data should be split chronologically rather than randomly to avoid using future information during training.

8. Performance Analysis

Regression metrics can include:

- MAE
- MSE
- RMSE
- MAPE

A plot comparing **actual vs predicted prices** should also be generated.

9. Justification

LSTM is designed to handle sequential data and retain relevant historical information. BPTT allows the model to learn temporal relationships by propagating errors through the sequence. Therefore, an LSTM-based model is a suitable architecture for the proposed stock-price forecasting task.

Q5. Integrated Deep Learning Solution for Automatic Video Captioning

1. Understanding of the Industry Problem

A media streaming company wants to automatically generate captions for uploaded videos to improve accessibility.

The system must understand the visual information contained in a video and convert that information into a meaningful textual description.

The proposed solution combines:

1. **Transfer Learning for visual feature extraction**
2. **LSTM-based Encoder–Decoder for caption generation**

This integrated architecture directly follows the requirement given in the assessment.

2. Overall Architecture

Uploaded Video
↓
Frame Extraction
↓
Pre-trained CNN
↓
Visual Feature Extraction
↓
Feature Sequence
↓
LSTM Encoder
↓
Context Representation
↓
LSTM Decoder
↓
Dense + Softmax
↓
Generated Caption

3. Step 1 – Video Frame Extraction

The uploaded video is divided into a sequence of frames.

For example:

Video
↓
Frame 1
Frame 2
Frame 3
...
Frame N

Instead of processing every frame, representative frames can be sampled at regular intervals to reduce computational cost.

4. Step 2 – Transfer Learning for Visual Features

A pre-trained CNN such as **ResNet50 or EfficientNet** is used.

The final classification layer is removed because the objective is not to classify the image. Instead, the CNN acts as a **feature extractor**.

Each frame is converted into a feature vector:

Frame 1 → Feature Vector 1
Frame 2 → Feature Vector 2
Frame 3 → Feature Vector 3
...
Frame N → Feature Vector N

These vectors represent visual information such as:

- Objects
- Shapes
- Textures
- Scene information

5. Step 3 – LSTM Encoder

The sequence of visual features is provided to an LSTM encoder.

Feature 1 → Feature 2 → Feature 3 → ... → Feature N

↓

LSTM Encoder

↓

Context Representation

The LSTM learns temporal relationships between frames.

For example, a video may show:

Person → picks up → ball → throws → ball

The temporal sequence helps the system understand the action rather than treating each frame independently.

6. Step 4 – LSTM Decoder

The decoder converts the learned visual representation into text.

It starts with a special token:

<START>

and predicts words sequentially.

<START>

↓

A

↓

person

↓

is

↓

throwing

↓

a

↓

ball

↓

<END>

The output at each step is passed through a Dense layer and Softmax to predict the next word.

7. Complete Architecture

VIDEO

↓

Frame Sampling

↓

┌──────────────────────────────┐
│ Pre-trained CNN │
│ ResNet / EfficientNet │
└──────────────────────────────┘

↓

Visual Feature Vectors

↓

LSTM Encoder

↓

Context Representation

↓

LSTM Decoder

↓

Dense + Softmax

↓

Generated Caption

8. Training Strategy

A video-caption dataset containing videos and corresponding captions is required.

Training steps:

1. Extract representative video frames.
2. Preprocess frames.
3. Extract CNN features.
4. Create sequences of visual features.
5. Tokenize captions.
6. Add START and END tokens.
7. Train the LSTM encoder-decoder.
8. Validate the model.
9. Evaluate generated captions.

Transfer learning allows the CNN to reuse previously learned visual features.

The CNN can initially remain frozen. If sufficient domain-specific data is available, selected deeper CNN layers can later be fine-tuned.

9. Tools

The system can use:

- Python
- TensorFlow/Keras or PyTorch
- OpenCV for video processing
- NumPy
- Pandas
- GPU/CUDA
- NLP tokenization libraries

10. Performance Evaluation

The captioning system can be evaluated using:

- BLEU
- ROUGE
- METEOR
- CIDEr
- Human evaluation

Both visual correctness and language quality should be considered.

For example, a caption should correctly identify the objects and actions shown in the video while also producing grammatically meaningful text.

11. How the Complete System Produces Accurate Captions

The two major components have different responsibilities.

Transfer Learning CNN

Extracts meaningful visual information from video frames.

LSTM Encoder

Understands the sequence of visual features and captures temporal relationships.

LSTM Decoder

Converts the learned representation into a sequence of words.

Softmax Output
Selects the probability distribution over the vocabulary for the next word.
Therefore:
Visual Information
+
Temporal Information
+
Language Generation
↓
Meaningful Video Caption

12. Justification
The integrated architecture is suitable because video captioning requires both **visual understanding** and **sequence generation**.
Transfer learning reduces the need to train a CNN from scratch and provides strong visual representations. The LSTM encoder captures information across video frames, while the decoder generates the caption word by word.
This combination provides an end-to-end approach for converting video content into accessible textual captions.

Overall Conclusion
The five proposed solutions apply deep learning techniques to practical industry problems:

Question	Industry Problem	Main Technique	Key Benefit
Q1	Skin cancer detection	Transfer Learning	Reuses pre-trained visual features with limited labeled data
Q2	Smart-city road segmentation	DenseNet + PixelNet	Feature reuse and pixel-level segmentation
Q3	Multilingual chatbot	Bidirectional RNN + Encoder–Decoder	Uses contextual information from both directions
Q4	Stock-price forecasting	LSTM + BPTT	Learns temporal dependencies
Q5	Automatic video captioning	Transfer Learning + LSTM Encoder–Decoder	Combines visual understanding with language generation

The assessment rubric assigns **20% to understanding the industry problem, 25% to application of engineering concepts, 25% to solution design, 20% to use of tools, and 10% to analysis**. Therefore, each solution above explicitly covers all five evaluation areas.

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context

Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts

RUBRICS FOR CALCULATION:

		solve the problem			
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	20%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis